TP 5: Cluster et GPU

TP de Parallélisme

14 octobre 2026

Le but de ce TP est d'utiliser le cluster de l'université pour compiler et exécuter vos programmes sur CPU et sur GPU.

Tâche 1

Connectez-vous au cluster Baobab en utilisant OpenDemand (https://openondemand.baobab.hpc.unige.ch/) (nécessite l'activation de la double authentification sur Switch).

En utilisant l'interface :
- Accéder au terminal (Cluster - Shell Access)
- Uploader le script SLURM `job.sh` (Files - Home Directory - Upload)

Créez et éditez un fichier directement sur Baobab en utilisant `nano` ou `vim` (`nano` est recommandé si vous n'avez jamais utilisé `vim`), puis téléchargez-le sur votre machine.

Bonus (1)

Utilisez `tmux` pour gérer plusieurs terminaux dans une même fenêtre.

Bonus (2)

Accédez à Baobab depuis votre terminal avec `ssh` :
- Générez une paire de clés publique/privée ssh avec `ssh-keygen -t ed25519 -C "your_email@example.com"`. Celles-ci sont créées dans votre home (selon votre système d'exploitation), dans le dossier `.ssh`.
- Copiez votre clé publique (`.pub`) sur https://my-account.unige.ch/. L'activation peut prendre jusqu'à 20 minutes.
- Connectez-vous à Baobab : `ssh <user>@baobab.hpc.unige.ch`. L'utilisateur est celui utilisé sur le site de l'université.

Tâche 2

Copiez l'implémentation de Mandelbrot utilisant les algorithmes standards sur le cluster.

Modifiez le script SLURM pour :
- compiler votre programme avec `g++` (ajoutez `-std=c++20 -O3` et `-ltbb`, chargez les modules nécessaires).
- exécuter votre programme séquentiellement sur une image de 20000×15000 pixels, avec 300 itérations maximum et la fenêtre `-2.2 1.0 -1.5 1.5`.

Adaptez le temps maximal d'exécution du job pour permettre un meilleur usage des ressources. Vous pouvez consulter la durée réelle avec `sac -j <jobid>`.

Le programme prend les arguments suivants, dans cet ordre : `largeur` et `hauteur` de l'image (en pixels), nombre maximal d'itérations, $x_{\min}$, $x_{\max}$, $y_{\min}$ et $y_{\max}$ de la fenêtre du plan complexe, puis le nom du fichier BMP de sortie. Pour tester rapidement avant le run complet :

```
./main 4000 3000 300 -2.2 1.0 -1.5 1.5 mandelbrot.bmp
```

Récupérez ce fichier et ouvrez-le sur votre machine pour vérifier le résultat.

Notez le temps d'exécution de la simulation 20000×15000 : il servira de référence pour les tâches suivantes.

IMPORTANT : aucun code ne doit être exécuté sur le nœud de login.

Bonus

Créez un second script pour exécuter l'implémentation utilisant OpenMP (compilation avec `-std=c++20 -fopenmp`).

Tâche 3

Parallélisez sur GPU l'implémentation de Mandelbrot utilisant les algorithmes standards.

Adaptez votre script pour avoir accès à un GPU et exécutez votre code, puis comparez le temps d'exécution avec ceux obtenus précédemment.

Indices : demandez un GPU avec `#SBATCH --gpus=1` et la partition `shared-gpu` (`#SBATCH --partition=shared-gpu`). Chargez le module `NVHPC` et compilez avec `nvc++ -std=c++20 -stdpar=gpu -O3` (algorithmes standards) ou `nvc++ -std=c++20 -mp=gpu -O3` (OpenMP).

Note : le code CPU ne se compile pas tel quel pour le GPU. Les points suivants doivent être adaptés :
- la fonction `std::lround()` n'est pas utilisable dans du code GPU.
- les variables locales `static` ne sont pas accessibles depuis du code GPU.
- une lambda exécutée sur le GPU ne peut pas capturer un `std::vector` par référence.

Vérifiez visuellement que le résultat est le même qu'avec le CPU.

Bonus

Portez l'implémentation OpenMP sur GPU.

Tâche 4

Avec les paramètres suivants : 20000×15000 pixels, 300 itérations maximum et la fenêtre `-2.2 1.0 -1.5 1.5`, exécutez l'implémentation CPU utilisant les algorithmes standards sur le cluster avec 1, 2, 4, 8 et 16 threads.

Observez l'évolution du temps d'exécution. Ce test est appelé le *strong scaling* : la taille du problème reste constante alors que le nombre de processeurs évolue.

Tâche 5

Reprenez la taille de problème de la Tâche 4 (20000×15000 pixels, avec les mêmes paramètres) pour l'implémentation GPU utilisant les algorithmes standards.

Exécutez cette implémentation sur le cluster sur GPU. Variez le **nombre de pixels** d'un facteur 0.5, 1 et 1.5.

Observez l'évolution des temps d'exécution. Ce test se rapproche du *weak scaling* : on fait varier la taille du problème sur une ressource fixe.